TP 1 : Compute shaders, fondamentaux

Addition vectorielle, Game of Life, réduction parallèle

Programmation parallèle GPU : OpenGL 4.3 / GLSL 4.30

Objectifs

- écrire, compiler et lancer un *compute shader* ;
- transférer des données entre CPU et GPU via des SSBO ;
- maîtriser les indices d'invocations et le lancement d'une grille de groupes de travail (*dispatch*) ;
- gérer une taille de donnée non multiple de la taille locale ;
- mettre en œuvre le *double buffering* (ping-pong) ;
- synchroniser des invocations d'un même groupe avec `barrier()` autour de la mémoire shared ;
- comparer deux stratégies de réduction et mesurer des temps GPU.

Prérequis

C++11 (tableaux dynamiques, fonctions), compilateur gcc, CMake, OpenGL moderne (création d'un contexte, compilation de shaders), notion de complexité ($O(n)$, $O(n^2)$).

1 Environnement et éléments fournis

Chaque exercice a son propre dossier, compilé en C++11 avec gcc et CMake (`cmake -S tp1/exercice1 -B tp1/exercice1/build` puis `cmake --build tp1/exercice1/build`, et de même pour `exercice2` et `exercice3`). Ces squelettes fournissent :

- la création de la fenêtre et du contexte OpenGL 4.3 *core profile* ;
- la compilation et l'édition de liens des shaders (loadComputeProgram("fichier.comp")) ;
- des utilitaires de création de buffers, décrits ci-dessous ;
- la mesure du temps se fait avec un simple chronomètre C++ (`std::chrono`), avec un `glFinish()` avant la lecture de l'horloge.

Interface du squelette

```
// Cree un SSBO de `bytes` octets, le remplit optionnellement
// avec `data`, et l'attache au slot `binding`.
GLuint makeSSBO(GLuint binding, GLsizeiptr bytes,
                const void* data = nullptr);

// Recupere le contenu d'un SSBO vers le CPU (lecture bloquante).
void readSSBO(GLuint buffer, GLsizeiptr bytes, void* dst);

// Division entiere arrondie vers le haut.
GLuint ceilDiv(GLuint n, GLuint d);
```

Attention

Une lecture bloquante (readSSBO) sert à *valider* les résultats. Pour *mesurer*, chronométrez en C++ (std::chrono) autour du seul glDispatchCompute, suivi d'un glFinish(), sans transfert CPU dans la boucle de mesure.

2 Conventions et notations

— **Invocation** : une exécution du shader. **Groupe de travail** : ensemble d'invocations partageant la mémoire shared et les barrières.

— **SSBO** : buffer OpenGL lisible et écrivable par les shaders, déclaré avec layout(std430) et relié à un slot via binding.

— **État courant / état suivant** : toute simulation du TP utilise deux buffers distincts, l'un lu, l'autre écrit.

— Les tailles locales des groupes sont déclarées *dans le shader* (layout(local_size_x = ...)), pas au lancement.

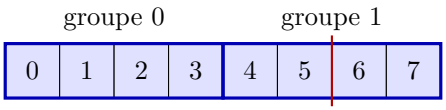
Rappel de synchronisation entre hôte et GPU :

Le lecteur suivant	**Bits de glMemoryBarrier**
un autre shader via SSBO	GL_SHADER_STORAGE_BARRIER_BIT
le CPU (glGetBufferSubData)	GL_BUFFER_UPDATE_BARRIER_BIT
le rendu via attributs de sommets	GL_VERTEX_ATTRIB_ARRAY_BARRIER_BIT

La barrière décrit *l'accès suivant*, jamais l'écriture précédente.

3 Exercice 1 : Addition vectorielle

Squelette : `tp1/exercice1`. Le shader `shaders/vecadd.comp` et la fonction `vecADD` sont à compléter.



n éléments : une garde empêche les invocations d'indice $\geq n$
de lire ou d'écrire un SSBO

Écrire la fonction

```
void vecADD(const float* A, const float* B, float* C, int n);
```

qui calcule, pour $0 \leq i < n$:

$$C_i = A_i + B_i. \tag{1}$$

3.1 Travail demandé

1. Créer trois SSBO (slots 0, 1, 2), transférer A et B, lancer le compute shader ci-dessous complété, récupérer C, libérer les buffers.
2. Compléter le shader :

```
#version 430 core
layout(local_size_x = 1024) in;

layout(std430, binding = 0) readonly buffer InputA {
    float A[];
};
layout(std430, binding = 1) readonly buffer InputB {
    float B[];
};
layout(std430, binding = 2) writeonly buffer OutputC {
    float C[];
};
uniform uint n;

void main() {
    // TODO : identifiant global, garde, addition
}
```

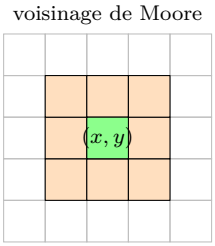
3. Écrire la version hôte : sélection du programme, transmission de n en uniform, calcul du nombre de groupes ($\lceil n/1024 \rceil$), dispatch, barrière adaptée, lecture.

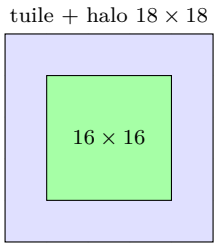
3.2 Questions

1. Quelle variable GLSL intégrée associe une invocation à un élément du tableau ? Donnez son expression développée en fonction de gl_WorkGroupID et gl_LocalInvocationID.
2. Pour $n = 100003$, combien de groupes lancez-vous ? Combien d'invocations sont lancées au total ? Combien sont inutiles ?
3. Pourquoi une invocation *inutile* ne doit-elle jamais lire ni écrire dans un SSBO ?
4. Une barrière barrier() entre invocations est-elle nécessaire ici ? Justifiez à partir des dépendances entre invocations.
5. Le shader utilise readonly et writeonly. Quel engagement prenez-vous en les écrivant ? Que se passe-t-il si un shader les viole ?

4 Exercice 2 : Conway, Game of Life

Squelette : `tp1/exercice2`. Il alloue les deux grilles, enchaîne le compute `life.comp` et affiche l'état courant.





Le monde est une grille de width × height cellules. Chaque cellule est un octet (uint8_t) : 0 (morte) ou 1 (vivante). Une génération est calculée à partir de la précédente selon les règles usuelles de Conway, appliquées au voisinage de Moore (les 8 cellules adjacentes) :

— une cellule vivante survit si elle a **2 ou 3** voisines vivantes ;

— une cellule morte naît si elle a **exactement 3** voisines vivantes ;

— dans tous les autres cas, la cellule est morte à la génération suivante.

4.1 Version A : calcul direct

Une invocation calcule une cellule de sortie. Utiliser deux SSBO en ping-pong :

```
#version 430 core
#extension GL_EXT_shader_8bit_storage : require
layout(local_size_x = 16, local_size_y = 16) in;

layout(std430, binding = 0) readonly buffer CurrentGrid {
    uint8_t current[];
};
layout(std430, binding = 1) writeonly buffer NextGrid {
    uint8_t next[];
};

uniform uint width;
uniform uint height;

void main() {
    // TODO : coordonnees (x, y) de l'invocation,
    //        garde, comptage des voisines, regles, ecriture
}
```

1. Comment convertir les coordonnées 2D de l'invocation en indice linéaire dans le buffer ?
2. Dans un premier temps, toute cellule hors de la grille est considérée comme *morte*. Écrire le comptage des voisines.
3. Écrire l'application des règles et la sortie dans next.
4. Côté hôte : écrire la boucle des générations. Expliquer pourquoi on *échange les rôles* des deux buffers au lieu de recopier next dans current.

Attention

Pendant un dispatch, current doit rester strictement en lecture et next strictement en écriture. Si une invocation lit une cellule que une autre vient de modifier, le résultat dépend de l'ordre d'exécution, non défini entre invocations.

4.2 Version B : conditions aux limites périodiques

Ajouter un mode *torique* : le bord droit est voisin du bord gauche, le bord haut du bord bas. Les coordonnées voisines sont ramenées dans la grille par modulo.

Vérifiez qu'une grille 3×3 possède bien huit voisines distinctes.

4.3 Version C : approfondissement, tuile avec halo

Chaque groupe calcule une tuile de 16×16 cellules de sortie. Pour compter les voisines sans relire le buffer global, chaque groupe charge en mémoire shared une zone de 18×18 cellules : la tuile plus un *halo* d'une cellule de large.

```
shared uint8_t tile[18][18];
```

1. Quels indices globaux correspondent aux cases de tile ?
2. Le chargement est *coopératif* : répartir les 324 chargements entre les 256 invocations du groupe (certaines invocations chargent deux cases). Écrire cette boucle.
3. Où placer la (ou les) barrière(s) barrier() ? Justifier.
4. Pourquoi une invocation située hors de la grille doit-elle participer au chargement et à la barrière, alors qu'elle ne produit aucune cellule de sortie ?
5. Combiner cette version avec les limites périodiques de la version B.

Attention

Toutes les invocations d'un groupe doivent rencontrer les barrières collectives, dans le même ordre. Un return précoce avant une barrier() est un comportement indéfini : placez la garde *après* le chargement et les barrières, autour du seul calcul de sortie.

4.4 Tests de validation

— un bloc carré de 2×2 cellules vivantes : doit rester stable (chaque cellule a exactement 3 voisines vivantes) ;

— un oscillateur de période 2 (clignotant) : doit alterner ;

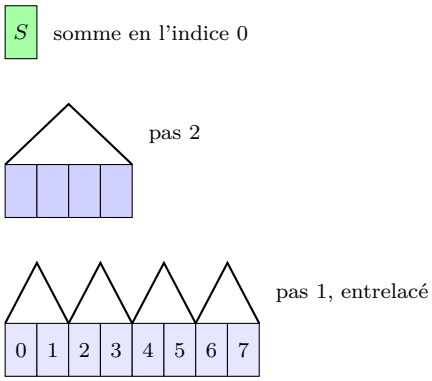
— un planeur (*glider*) : doit se déplacer en diagonale d'une case toutes les 4 générations ;

— une initialisation aléatoire de densité 0,3 : vérifier que le nombre total de cellules vivantes évolue de façon plausible (exercice 3).

L'affichage se fait *sans* rapatrier la grille sur CPU : un fragment shader lit l'état courant dans le SSBO et l'affiche.

5 Exercice 3 : Réduction parallèle

Squelette : `tp1/exercice3`. La fonction `reduce` et le shader `reduce.comp` sont à compléter.



On veut calculer $S = \sum_{i=0}^{n-1} x_i$ pour un vecteur de taille n puissance de deux, avec $1 \leq n \leq 1024$, sur *un seul groupe de 1024 invocations*. Le résultat est écrit dans l'élément d'indice 0 du SSBO. Les invocations d'indice $\geq n$ chargent 0.

Interface hôte :

```
void reduce(const float* vec, float* sum, int size);
```

Déclarations communes :

```
#version 430 core
layout(local_size_x = 1024) in;

layout(std430, binding = 0) buffer Vector {
    float values[];
};

uniform uint size;

shared float partial[1024];
```

5.1 Version 1 : réduction entrelacée

À chaque étape, les invocations actives additionnent des éléments de plus en plus espacés :

pas 1 : $x_0{+}x_1$, $x_2{+}x_3$, $x_4{+}x_5$, …
pas 2 : $x_0{+}x_2$, $x_4{+}x_6$, …
pas 4 : $x_0{+}x_4$, …

1. Charger les données dans `partial`, en complétant par zéro au-delà de `size`.
2. Écrire la boucle de réduction.
3. Placer les barrières nécessaires. Pourquoi sont-elles indispensables ici, alors qu'elles étaient inutiles à l'exercice 1 ?
4. Décrire le motif des invocations actives à chaque étape : que remarque-t-on sur leur répartition au sein du groupe ?

5.2 Version 2 : réduction par moitiés

Réaliser une seconde version où les invocations actives forment un intervalle contigu :

pas $n/2$: $x_t{+}x_{t+n/2}$ pour $t < n/2$
pas $n/4$: $x_t{+}x_{t+n/4}$ pour $t < n/4$
… jusqu'au pas 1 : $x_0{+}x_1$

1. Écrire cette version.
2. Comparer les motifs d'activation des deux versions : laquelle garde des invocations actives *regroupées* le plus longtemps ?
3. Mesurer les deux versions pour $n = 1024$ (moyenne sur 500 itérations). Le classement observé est-il stable ?

Attention

Les barrières doivent être placées *hors* des conditions `if` de la réduction : même une invocation inactive (qui n'additionne rien) doit rencontrer chaque `barrier()`. Justifiez ce point.

5.3 Validation

Comparer avec une somme CPU. Pour des entrées aléatoires positives puis positives et négatives, expliquer pourquoi l'égalité *bit à bit* avec la somme CPU n'est pas un critère approprié pour toutes les entrées (arrondi flottant, ordre des additions).

5.4 Approfondissement : réduction multi-passes

Étendre le calcul à un n quelconque (n jusqu'à 10^6) :

1. chaque groupe calcule une somme partielle dans un buffer de résultats intermédiaires ;
2. un second dispatch réduit ces sommes partielles, et ainsi de suite jusqu'à un résultat unique.

Justifier pourquoi ces étapes doivent être des dispatchs séparés, avec une barrière SSBO entre chacun, alors qu'il n'existe pas de barrière globale *à l'intérieur* d'un shader.

5.5 Application : comptage des cellules vivantes

Utiliser la réduction pour afficher à chaque génération du Game of Life le nombre de cellules vivantes. En extension, calculer le nombre de cellules ayant changé d'état entre deux générations :

$$D_t = \sum_i \left[\mathrm{etat}_i^{t+1} \neq \mathrm{etat}_i^{t} \right]. \tag{2}$$

Interpréter le cas $D_t = 0$. Montrer sur un exemple (oscillateur de période 3) qu'il ne suffit pas à détecter tous les comportements périodiques.

6 Livrables

Pour chaque exercice : les shaders, la fonction hôte, l'implémentation CPU de référence, les tests demandés, la justification des synchronisations, les mesures. Les approfondissements (version C du Game of Life, réduction multi-passes) sont clairement séparés des versions de base.